

Secure Chat Application with Encryption

CNS course project — Team 15.

A client-server chat application in which two peers exchange messages that the relay server and any network observer can never read in plaintext: each session starts with an RSA-signed ECDH (X25519) handshake between the two peers (never trusting the server to broker keys), derives a fresh AES-256-GCM session key, signs and encrypts every message with it, wraps the whole client↔server connection in TLS, gates every connection behind a PBKDF2-hashed login, rejects replayed or tampered messages, detects a relay dropping or reordering messages via a hash-chained conversation log, and locks out repeated failed logins — all properties backed by 160 automated tests and five live attack-and-defense demo scripts, plus a Tkinter GUI whose "Wire Log" panel makes the cryptography visible during a demo instead of invisible, an opt-in **Attack Lab** that triggers those same attacks for real with one click, and a live **Security Dashboard**.

PDF copies of this documentation are available in [docs/pdf/](#) for offline viewing or printing.

How this differs from WhatsApp/Signal

WhatsApp and Signal are built to be **deniable**: after a conversation, nobody can prove to a third party who wrote what, and that is a feature for private messaging. This project makes the opposite, deliberate trade: an **auditable secure messenger** where every message is **provably attributable**, the conversation as a whole is **tamper-evident**, and the evidence can be **verified offline by someone who trusts neither user nor server**.

	WhatsApp / Signal	This project
Who wrote it	Deniable by design: no transferable proof of authorship	Non-repudiation : every message is signed with the sender's long-term RSA key; anyone holding the public key can verify it
Relay drops or reorders a message	Each message is authentic on its own; conversation integrity isn't a surfaced, auditable guarantee	Detected : a per-sender hash chain (sequence number + hash of the previous message, inside the signed payload) flags "message(s) missing, possible deletion by the relay"
Evidence for a third party	None: nothing exportable that proves anything	Signed evidence file plus a standalone verifier (<code>tools/verify_transcript.py</code>) that imports no client or server code
What the user can see	Opaque; at most a safety number	Security receipt on every message (click a bubble) and a live Wire Log of what actually crosses the network
Live attacks, on demand	Not applicable (no equivalent feature)	Attack Lab : a GUI panel that arms a real relay attack (tamper/replay/drop/MITM) for the next message, caught live by the other window's real, unmodified checks
Account abuse / intrusion visibility	Not applicable	Login lockout with exponential backoff, and a live Security Dashboard showing event counts, a colour-coded feed, and currently-locked accounts

Target use cases: regulated environments where "who said what, and was anything removed?" must have a defensible answer: healthcare instructions (a dosage order, and proof that a later "stop" order was not silently dropped), financial approvals, legal and compliance communication, and command chains where orders must be attributable and complete. It is **not** the right tool where plausible deniability matters (e.g. whistleblowing or dissent): there, use Signal.

Making attacks and defenses visible from inside the app itself (not just in separate scripts) is the other half of this: the opt-in **Attack Lab** (see "Running the Attack Lab + Security Dashboard" below) lets the team trigger a real relay attack with one click during a live demo and watch the other window's real client-side checks catch it, and the **Security Dashboard** gives a continuously-updated view of every login failure, lockout, and detected attack across the whole server.

Quick start (clean clone → working two-GUI demo)

On a Mac, one command: double-click [demo/start_demo.command](#) in Finder (or run `demo/start_demo.sh` in a terminal). It stops any leftover server of ours, checks the TLS certificate, then opens clearly titled Terminal windows for the server (Attack Lab on), two chat windows and the security dashboard, and waits until the server is really accepting connections before launching them. Options: `--no-lab` (normal mode) and `--port N`. Stop everything again with `demo/stop_demo.sh`, which touches only this project's own processes (never macOS AirPlay, which also listens on port 5000).

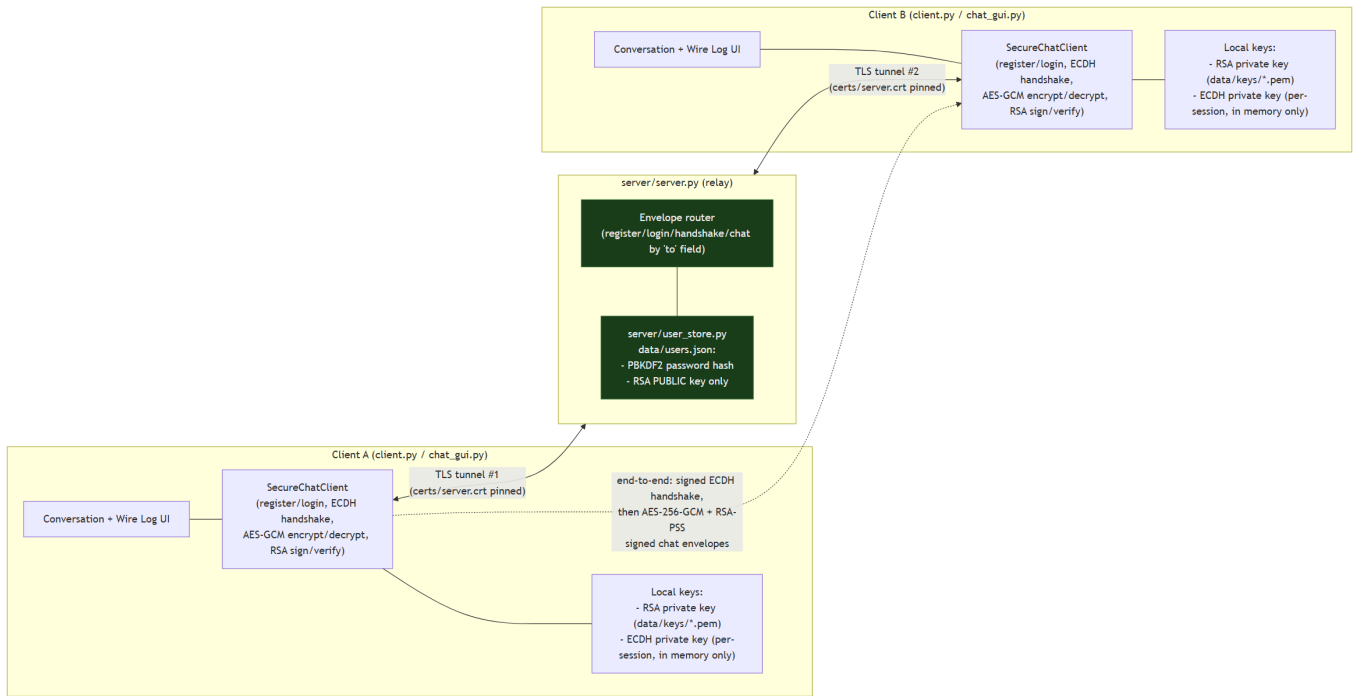
Or step by step:

```
git clone <this repo's URL>
cd CNSProject
pip install -r requirements.txt
python certs/generate_certs.py      # one-time: generates certs/server.crt + server.key
python server/server.py             # terminal 1 – leave this running
python gui/chat_gui.py              # terminal 2
python gui/chat_gui.py              # terminal 3
```

In each GUI window: fill in a username, the *other* window's username as the peer, and a password, then click **Register** (the first time) or **Login** (afterward). Once both windows show a fingerprint at the top, type a message and click **Send** — it appears in the **Conversation** panel on both sides, and its encrypted form appears in the **Wire Log** panel. For the exact, click-by-click version of this (what a teammate who's never run it before should do, live in front of the professor), see [demo/full_walkthrough.md](#). Click any message bubble to open its **security receipt**, and use **Export Evidence** (top bar) to write a signed evidence file you can check with `python tools/verify_transcript.py`.

Architecture

Two clients, one relay server, TLS wrapping every connection, and an inner end-to-end layer (ECDH + AES-GCM + RSA) the server itself cannot see into:



See [docs/architecture.md](#) for the full-resolution diagram, the Mermaid source, and a breakdown of exactly what the server can and cannot see at each point.

Security properties

Every row names the module that implements the property and a real, currently-passing test that exercises it (verified — see "Running the tests" below):

Property	What guarantees it	Module	Real test
Confidentiality	AES-256-GCM encrypts every chat message with a per-session key the server never holds; TLS additionally wraps the whole connection	crypto_engine/aes_gcm.py , certs/ + connect_tls() in client/client.py	tests/test_aes_gcm.py::test_ciphertext_is_not_plaintext
Integrity	AES-GCM's authentication tag detects any bit-flip in transit; a compromised relay that tampers a message cannot get it accepted	crypto_engine/aes_gcm.py	tests/test_aes_gcm.py::test_tampered_ciphertext_is_rejected
Authentication	The ECDH handshake itself is signed with each side's RSA identity key and verified before any session key is derived, stopping a relay from substituting its own key mid-handshake	crypto_engine/signatures.py , signed-handshake logic in client/client.py	tests/test_handshake_auth.py::test_handshake_signed_with_wrong_key_is_rejected

Property	What guarantees it	Module	Real test
Non-repudiation	Every chat message is signed with the sender's long-term RSA private key; a message can't be forged as if signed by someone else's key	crypto_engine/signatures.py	tests/test_signatures.py::test_wrong_public_key_fails_verification
Key management	A brand-new ECDH keypair is generated for every session (forward secrecy — compromising one session's key reveals nothing about any other); each identity's long-term RSA private key never leaves its own machine	crypto_engine/dh_exchange.py , auth/keystore.py	tests/test_dh_exchange.py::test_fresh_keypairs_produce_different_session_keys
Access control	A connection is not admitted to the roster or allowed to route any handshake/chat envelope until it completes a successful PBKDF2-checked login	auth/password_hash.py , server/server.py (_authenticate)	tests/test_tls_setup.py::test_tls_client_wrong_password_login_rejected
Integrity of conversation	A per-sender SHA-256 hash chain (seq + prev_hash , inside the signed and encrypted payload) lets the receiver detect a relay silently dropping, reordering or injecting messages	crypto_engine/hash_chain.py , chain check in client/client.py	tests/test_hash_chain.py::test_real_pipeline_detects_relay_dropping_a_message
Evidence	Both directions of a session can be exported as one JSON file, signed by the exporter, and checked offline by an independent verifier (signatures, both hash chains, chain heads, export signature)	client/evidence.py , tools/verify_transcript.py	tests/test_evidence_export.py::test_editing_one_message_makes_it_invalid_and_names_that_mess

Property	What guarantees it	Module	Real test
Attack Lab containment	A <code>lab_control</code> request is honored only when the server was started with <code>--lab</code> , only from a localhost peer, and only from an already-authenticated connection; any other attempt is rejected and logged, never silently ignored	<code>server/server.py</code> (<code>_handle_lab_control</code>)	<code>tests/test_attack_lab.py::test_lab_control_rejected_from_non_localhost_peer</code>
Login lockout	After repeated failed logins, an account is locked with exponential backoff; a locked attempt is rejected <i>before</i> PBKDF2 runs, so hammering a locked account cannot be used to burn CPU	<code>server/lockout.py</code>	<code>tests/test_security_events.py::test_full_lockout_flow_end_to_end</code>

Run any single one directly, e.g. `pytest tests/test_aes_gcm.py::test_ciphertext_is_not_plaintext -v`.

Known limitations

Said out loud, not buried — these are the honest edges of this project's scope:

- **The Attack Lab has no one-click brute-force/lockout demo button.** Lockout is real and fully tested (`tests/test_lockout.py`, and `tests/test_security_events.py::test_full_lockout_flow_end_to_end` end to end through a real server, including that a locked attempt skips PBKDF2 entirely), but the GUI does not expose a button that fires repeated login attempts at a named account, even against one's own test account. See it live instead by running a handful of deliberate wrong-password **Login** attempts in a GUI window.
- **Login lockout state is in memory only**, per `server/lockout.py`'s docstring: a server restart clears every lock and failure count. Acceptable for a single course-project relay server; a production system would persist it.
- **The register/login password is not protected beyond TLS.** The AES-GCM/RSA layer only exists between two chat *peers*, after a handshake; it doesn't cover the client↔server login exchange, which relies entirely on TLS. If the server itself were compromised, it would see plaintext passwords (never plaintext chat content, and never anyone's private key) at the moment of login, before hashing. See [docs/architecture.md](#).
- **RSA private keys are stored unencrypted on disk** (`data/keys/<username>_private.pem`). A production system would wrap this file under a key derived from the user's password; this project deliberately scoped that out — see the comment in `auth/keystore.py`.
- **No account recovery.** If a user's private key file is lost, their account cannot send verifiable messages again without re-registering under a new username — there is no re-keying or password-reset flow.
- **Self-signed cert trust is manual and local.** Each teammate must run `certs/generate_certs.py` and the *same* generated `certs/server.crt` must be present on every machine the server and clients run on; there's no certificate authority or distribution mechanism beyond "share the file."
- **The server trusts a client's claimed `from` field for routing.** It does not cryptographically bind a connection's authenticated username to the `from` field on every envelope that connection sends — the handshake/message signatures are what actually prevent forgery (the receiver always verifies against the *real* sender's RSA key, not whatever the envelope claims), but this is worth knowing rather than assuming the server enforces it.
- **Hash chains are per direction.** Each sender's messages are chained to each other, so a dropped, reordered or edited message is caught. The *interleaving* of alice's and bob's messages relative to each other is not cryptographically bound (timestamps give only a rough order).
- **Evidence proves consistency, not identity, unless you pin a fingerprint.** Anyone can fabricate a complete, self-consistent evidence file with their own keys. An auditor must obtain a participant's key fingerprint out of band and pass it with `--expect-fingerprint`; the verifier then rejects any file that doesn't contain it.
- **Non-repudiation is the opposite of deniability.** By design, a participant cannot later deny having sent a signed message. That is the point for audit use cases, and a drawback for anything needing deniability.
- **Single relay server, no redundancy, no persistence beyond a local JSON file** — this is a course project demonstrating the cryptography, not a deployable service (no horizontal scaling, no database, no rate limiting, no replay-window synchronization across multiple server instances).
- **Wireshark capture is a single short session** (`demo/capture_normal_session.pcapng`), not a comprehensive traffic sample — see `demo/capture_instructions.md` if you want to record a longer one.

Live demo scripts

`demo/run_*.py` are standalone scripts that each start their own real mini relay server and two real clients to demonstrate one attack being stopped — meant to be run live for a demo or in front of the professor, not just as passing tests. Each prints a clear `[PASS] / [FAIL]` per check and an overall result:

Script	What it proves
<code>demo/run_tamper_demo.py</code>	A malicious/compromised relay flipping a ciphertext byte in transit cannot get a tampered message displayed — AES-GCM's tag catches it.
<code>demo/run_replay_demo.py</code>	A relay resending a captured chat message a second time cannot get it displayed twice — the timestamp + duplicate-nonce tracking rejects the replay.
<code>demo/run_mitm_handshake_demo.py</code>	A malicious relay substituting its own ECDH public key during the handshake is stopped by the signed handshake — no session key is ever derived, and the fingerprint that would result doesn't match either.
<code>demo/run_drop_demo.py</code>	A relay silently dropping one valid message mid-conversation is caught: the receiver sees message #3 where #2 was expected and warns "possible deletion by the relay".
<code>demo/run_evidence_demo.py</code>	A real conversation is exported as signed evidence and verified (VALID) by the independent verifier; then one word in one message of a copy is changed and the verifier reports INVALID, naming that message.

```
python certs/generate_certs.py # once, if you haven't already
python demo/run_tamper_demo.py
python demo/run_replay_demo.py
python demo/run_mitm_handshake_demo.py
python demo/run_drop_demo.py
python demo/run_evidence_demo.py
```

See [demo/README.md](#) for more detail, and [demo/full_walkthrough.md](#) for the full click-by-click live demo script (GUI + these demo scripts + the Wireshark capture + the Attack Lab and Security Dashboard, with talking points and timing).

Running the Attack Lab + Security Dashboard

1. **Start the server with `--lab`** (off by default -- a server started without it behaves exactly as before, and rejects and logs any `lab_control` request it receives):

```
python server/server.py --lab
```

2. **Open the security dashboard**, in its own window, any time (it only tails `logs/security_events.jsonl` -- it never connects to the server):

```
python gui/security_dashboard.py
```

3. **Open two GUI clients** as in "Quick start" above. Once a secure session is established, each window shows an **Attack Lab** button (amber, top bar) -- it's only enabled when the connected server reports lab mode. Click it to open the panel (the amber "LAB MODE" banner is deliberate, so it's never mistaken for a real error), then click **Tamper next message**, **Replay last message**, **Drop next message**, or **MITM next handshake**. Each one is a real, one-shot attack carried out by the relay server itself -- never simulated -- and the *other* window's real, unmodified checks (AES-GCM tag, replay window, hash chain, or handshake signature) catch it live, exactly as the standalone `demo/run_*.py` scripts demonstrate, with the dashboard's event feed updating in real time alongside it.

Login lockout triggers automatically: five wrong passwords for one username within five minutes lock that account with exponential backoff (30s, 60s, 120s, ... capped at 15 minutes); the dashboard's "Locked accounts" panel shows it and the remaining time. A locked login is rejected *before* PBKDF2 ever runs (see `server/lockout.py`), so this cannot be used to burn CPU against an account that's already locked.

Running the tests

```
pip install -r requirements.txt
pytest tests/ -v
```

160 tests across 15 files (the first seven one per project phase): `test_aes_gcm.py` (encryption), `test_dh_exchange.py` (key exchange), `test_password_hash.py` (login), `test_signatures.py` (non-repudiation), `test_handshake_auth.py` (signed-handshake MITM fix), `test_replay_protection.py` (replay defenses), `test_tls_setup.py` (transport security), plus `test_hash_chain.py` (dropped/reordered-message detection), `test_evidence_export.py` (signed export + offline verifier, including tampered files), `test_attack_lab.py` (opt-in lab-mode relay attacks and their safety restrictions), `test_lockout.py` (login lockout + per-address rate limiting), `test_security_events.py` (the security event log end to end, including the full lockout flow and that a locked attempt never runs PBKDF2), `test_entrypoints.py` (the documented ways of starting the project, run as real scripts), `test_gui_layout.py` (header, input row, login card and dashboard fit their windows; skipped automatically without a display) and `test_reset_environment.py` (the reset script only ever kills our own `server.py`).

The server-based tests are deterministic: each starts its server on an OS-chosen port, waits on real conditions instead of sleeping, tears everything down afterwards, and gets its own lockout state, user store, key directory and event log, so no test touches your real `data/`. (An earlier intermittent timeout was traced to a real bug -- sharing one TLS socket between threads -- fixed in `transport/locked_tls.py`.)

How to run (detail)

Requires Python 3.8+.

0. **One-time per teammate: generate a local TLS certificate.** The server and client both need `certs/server.crt` / `certs/server.key`, which are gitignored (a committed private key would let anyone with repo access impersonate the server) and must be generated locally, once, by each person running this project:

```
python certs/generate_certs.py
```

This writes a self-signed cert for `CN=localhost`, valid 365 days. Re-run with `--force` to regenerate (then restart the server and re-run this on every client, too — an old cert and a new one won't match).

1. **Start the server** (one terminal):

```
python server/server.py
```

Defaults to `127.0.0.1:5000`; override with `--host / --port`. It prints `Server listening on 127.0.0.1:5000 over TLS ...` once ready.

2a. Terminal client — start two clients, each naming the other as its peer (one in each of two more terminals):

```
python client/client.py --peer bob
python client/client.py --peer alice
```

Each client first asks you to **register** a new account or **login** to an existing one — pick `1) Register` the first time for each username, `2) Login` afterward. The password prompt hides your input (it won't echo to the terminal).

2b. GUI client — see "Quick start" above, or `demo/full_walkthrough.md` for the click-by-click version.

Registering also generates a long-term RSA-2048 identity keypair for that account (see "Design details" below) — the private key is saved locally, and only the public key is sent to the server. Once both peers are authenticated and online, they automatically perform the signed ECDH handshake, fetch each other's RSA public key, and from then on every message is signed, AES-256-GCM encrypted before it leaves the client, and decrypted + signature-verified on arrival.

Registered accounts (PBKDF2 password hashes and RSA public keys only) persist in `data/users.json`; each account's RSA private key is saved separately as `data/keys/<username>_private.pem`. Both `data/users.json` and `data/keys/` are gitignored since they hold real credentials and private key material from local testing.

Troubleshooting: certificate mismatch

Symptom: a client fails to connect with something like `[!] Server certificate verification failed -- refusing to connect. ([SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed: self-signed certificate ...)`.

Cause: `certs/server.crt` was regenerated (e.g. `python certs/generate_certs.py --force`) while an *old* `server.py` process was still running — that process loaded its certificate into memory at startup and keeps using it until restarted, so it's now serving a different cert than the one on disk that clients trust.

How to tell at a glance (no guessing required): both the server and any connecting client print a short cert fingerprint — the server at startup (`Server listening on ... (cert fingerprint: XXXX XXXX XXXX XXXX)`) and the client right before it attempts the handshake (`[tls] trusting server cert fingerprint: XXXX XXXX XXXX XXXX` — shown in the terminal client's output, or in the GUI's **Wire Log** panel). If these two don't match, that's the stale-server bug, confirmed instantly instead of debugged manually.

Fix:

```
python demo/reset_environment.py
```

This detects a leftover process on the server's port (asking before killing anything — it never kills silently), regenerates `certs/server.crt / server.key`, and prints the new fingerprint. Restart the server (`python server/server.py`) and confirm its startup line shows that *same* fingerprint before launching any clients. It does not touch `data/users.json` or `data/keys/` — registered accounts and identity keys are unaffected by a cert reset.

Or manually: stop the server (Ctrl+C its terminal), regenerate certs (`python certs/generate_certs.py --force`), then start the server again *before* starting any clients.

Wire protocol

One JSON object per line (newline-terminated), carried inside the TLS tunnel. Every envelope has a `"type"` field:

type	direction	carries
<code>register</code>	client → server	<code>username</code> , <code>password</code> , <code>public_key</code> (RSA PEM text)
<code>register_result</code>	server → client	<code>success</code> , <code>reason</code>
<code>login</code>	client → server	<code>username</code> , <code>password</code>
<code>login_result</code>	server → client	<code>success</code> , <code>reason</code>
<code>roster</code>	server → client	list of currently-online usernames
<code>system</code>	server → client	human-readable join/leave/error text
<code>user_joined</code>	server → client	<code>username</code> of a newly-connected peer
<code>handshake_init</code>	client ↔ client (via server)	<code>from</code> , <code>to</code> , <code>pubkey</code> (base64, 32 raw bytes), <code>handshake_sig</code> (base64, RSA-PSS signature over <code>pubkey</code>)
<code>handshake_response</code>	client ↔ client (via server)	<code>from</code> , <code>to</code> , <code>pubkey</code> (base64, 32 raw bytes), <code>handshake_sig</code> (base64, RSA-PSS signature over <code>pubkey</code>)
<code>get_pubkey</code>	client → server	<code>username</code> (whose RSA public key to look up)
<code>pubkey_result</code>	server → client	<code>username</code> , <code>public_key</code> (RSA PEM text or null), <code>success</code>
<code>chat</code>	client ↔ client (via server)	<code>from</code> , <code>to</code> , <code>nonce</code> , <code>ciphertext</code> , <code>tag</code> (all base64); the plaintext AES-GCM decrypts to is itself <code>{"message", "timestamp", "seq", "prev_hash", "signature"}</code> JSON, where <code>signature</code> covers the canonical record <code>{sender, recipient, seq, timestamp, message, prev_hash}</code> (see "How the hash chain works")

A connection must complete a successful `register` or `login` exchange before the server admits it to the roster or accepts any handshake/chat envelope from it. The server routes `handshake_init / handshake_response / chat` envelopes to the named `to` recipient only, based on the sender's own claimed `from` field; it never inspects or needs to understand their payload beyond that routing. `get_pubkey` is answered directly by the server from its user store (the requested user doesn't need to be online — a public key is public information regardless of presence).

Design details

▼ How the handshake works (ECDH + forward secrecy)

Each client generates a brand-new X25519 (elliptic-curve Diffie-Hellman) keypair for every chat session — never reused across logins, so compromising one session's key reveals nothing about any other session (forward secrecy). When two named peers are both online, the client whose username sorts alphabetically first automatically sends its public key to the other, relayed through the server inside a ``handshake_init`` JSON envelope; the second client generates its own keypair and replies with a ``handshake_response`` carrying its own public key. Both sides then independently run ``compute_shared_key()`` — each combining "their own" private key with the "other's" public key — which by the math of Diffie-Hellman produces the identical raw shared secret on both ends without either private key ever crossing the network. That raw secret is then passed through HKDF-SHA256 to derive a clean, uniformly random 32-byte AES-256 key (the raw ECDH output itself is never used directly as a cipher key). From that point on, every chat message is encrypted with ``aes_gcm.encrypt()`` before being sent and decrypted with ``aes_gcm.decrypt()`` after being received; the relay server only ever forwards opaque public-key bytes and encrypted (nonce/ciphertext/tag) envelopes — it never computes, stores, or sees the shared secret, and a tampered or forged message is rejected with a warning instead of being decrypted into garbage.

▼ How the signed handshake stops a relay-server MITM

The handshake above has a gap: ``handshake_init``/``handshake_response`` carry a raw ECDH public key, and nothing about the handshake itself ties that key to a particular long-term identity. A malicious or "compromised relay server" — a different threat than a network eavesdropper; TLS does not help here, since the server is the trusted TLS endpoint — could substitute its own ECDH public key for either peer's, completing two separate handshakes (attacker↔A, attacker↔B) while both clients believe they're talking directly to each other. Since each client already has a long-term RSA identity, the fix reuses it: the ECDH public key is signed with the sender's RSA private key before sending (``handshake_sig``), and the receiver verifies that signature against the sender's already-known RSA public key "before" computing the shared secret. If verification fails — or the ECDH key was swapped after signing, or the signature was produced by the wrong RSA key — the handshake "aborts with a clear error and no session key is ever derived"; nothing gets silently trusted. After a "successful" handshake, both clients also print a short human-readable "fingerprint" of the peer's RSA public key (``crypto_engine/signatures.fingerprint``, the first 16 hex characters of SHA-256 of the key's PEM bytes) — the "check key fingerprints" mitigation: two people can read this aloud to each other (voice call, in person) to independently confirm they hold the same identity for their peer, catching a MITM even if some future bug let a forged handshake slip past the automated check. See ``demo/run_mitm_handshake_demo.py`` for a live demonstration.

▼ How replay protection works

A captured, validly-encrypted, validly-signed chat envelope could otherwise be resent later (or twice) and would still pass both AES-GCM and the RSA signature check, since neither says anything about "when" the message is being presented. Two independent defenses close this: first, a Unix timestamp is included in what gets "signed" (inside the canonical record built by ``crypto_engine/hash_chain.canonical_record``) before encryption, so it travels tamper-evident — an attacker can't just bump the timestamp on a captured message to make it look fresh again, since that invalidates the signature. On receipt, a message is rejected if its timestamp is more than ``REPLAY_WINDOW_SECONDS`` (30s) old or more than ``CLOCK_SKEW_SECONDS`` (5s) in the future. Second, belt-and-suspenders: each client tracks recently-seen (sender, nonce) pairs in memory (nonces are already fresh random values per ``aes_gcm.encrypt()`` call, so a genuine resend from the sender would carry a "different" nonce) and rejects an exact repeat outright, even if it somehow fell inside the freshness window; old entries are pruned automatically once they're old enough that the timestamp check alone would catch them anyway. See ``demo/run_replay_demo.py``.

▼ How the hash chain works (dropped / reordered message detection)

Per-message AES-GCM and RSA signatures prove each message is authentic, but they can't notice a message that "never arrives": a malicious relay can drop or reorder valid messages and every message that does arrive still verifies. So each sender keeps a chain over its own messages. Every outgoing message carries ``seq`` (1, 2, 3, ...) and ``prev_hash`` (the SHA-256 of that sender's previous canonical record; for the first message, a "genesis" value derived from both public ECDH keys of the handshake plus the sender's name, so no two sessions or directions share a start). Both fields sit inside the RSA-signed, AES-GCM-encrypted payload, so the relay can neither read nor alter them. The canonical record is ``canonical_json`` (where ``canonical_json`` is sorted-key, no-whitespace, ASCII-escaped JSON; the signature and the record hash are both taken over exactly those bytes (``crypto_engine/hash_chain.py``)). On receipt, after decrypt, signature, freshness and duplicate checks, the chain check runs: ``seq`` exactly as expected and ``prev_hash`` matching is "OK"; ``seq`` ahead of expected is a "gap" ("N message(s) missing, possible deletion by the relay": the authentic message is still shown and the chain resyncs, but a warning is raised); ``seq`` behind expected, or a wrong ``prev_hash``, is "broken" ("possible reordering, replay or injection": the message is not shown). Gaps and rejections are recorded and exported with the evidence. See ``demo/run_drop_demo.py``.

▼ How the signed evidence export and offline verifier work

Each client keeps an in-memory transcript of every message that passed all checks (both directions, with the sender's signature), plus a separate list of rejected events that is never mixed into it. "Export Evidence" (header button, or ``client.export_evidence()``) writes ``exports/evidence____.json`` (gitignored): participants with their RSA public keys and fingerprints, session start and end, the public signed handshake values, the ordered records, each side's final chain head, the rejected events, and the exporter's RSA signature over the canonical bytes of everything else. ``python tools/verify_transcript.py exports/json`` then checks, without importing any client or server code: fingerprints against keys, the handshake signatures, every message's signature, ``record_hash`` and chain link (recomputing both chains from genesis), the final chain heads, and the exporter's whole-file signature. It prints a per-message report and an overall "VALID" or "INVALID" that names the exact record and check that failed. ``--expect-fingerprint "XXXX XXXX XXXX XXXX"`` pins a key you trust out of band (see Known limitations). See ``demo/run_evidence_demo.py``.

▼ How signing works (non-repudiation)

The ECDH session key proves a message was encrypted for this pair of peers, but it's symmetric — both sides hold it, so it can never by itself prove "which" of them actually wrote a given message to a third party. A separate, long-term RSA-2048 keypair per user identity closes that gap: at registration a client generates its own RSA keypair, keeps the private key on disk locally (see ``auth/keystore.py``), and sends only the public key to the server, which stores it next to that user's password hash. Before sending a chat message, the sender signs the plaintext with its own RSA private key (RSA-PSS + SHA-256), then wraps {message, signature} together as the payload that gets AES-256-GCM encrypted — "sign, then encrypt" — so the signature travels protected inside the same ciphertext as the message, never sent in the clear. On the receiving side the order reverses: "decrypt, then verify" — the client AES-GCM-decrypts the envelope, unwraps {message, signature}, and only then checks the signature against the sender's RSA public key (fetched once via a ``get_pubkey`` request to the server and cached for the session). A message whose signature doesn't check out — tampered content, a forged signature, or a signature checked against the wrong public key — is never displayed; the client prints a clear warning and discards it.

▼ How TLS layers with the app-level crypto

TLS (``ssl.PROTOCOL_TLS_SERVER`` / ``PROTOCOL_TLS_CLIENT``) wraps the whole client↔server TCP connection, wrapping every socket "before" any envelope — including the very first register/login — is read or written. This is a "transport-layer" protection: it makes the connection itself opaque to anyone on the network path, so an observer between a client and the server can no longer see the JSON structure, field names, or content of anything at all, not even metadata like "type": "login". It sits alongside, not in place of, the "application-layer" crypto — AES-GCM session keys and RSA signatures — which protect message content and identity end-to-end between the two chat "peers themselves", independent of transport. The distinction that matters: TLS protects data only while it's in transit to/from the server; the server itself sees the connection in the clear once TLS terminates there (it's the trusted relay, not an untrusted network link — see "Known limitations" above for what that means for the login password specifically). AES-GCM + RSA signatures, by contrast, protect a chat message even from the server — the server only ever sees opaque ciphertext and routing metadata, never plaintext content, a private key, or a session key, whether or not TLS is present. The client does not use ``check_hostname=False`` or ``CERT_NONE`` to sidestep certificate verification — that would accept a certificate from literally anyone, which defeats the point of using TLS at all. Instead it pins trust to exactly the one self-signed certificate this project generated (``certs/server.crt``, loaded via ``load_verify_locations``), so a different server presenting a different certificate is rejected at the TLS handshake, before any envelope is ever sent.

▼ Crypto modules — quick reference

``crypto_engine/aes_gcm.py`` — standalone AES-256-GCM module:

```
from crypto_engine import generate_key, encrypt, decrypt

key = generate_key() # 32 random bytes (testing only)
```

```
box = encrypt(key, b"hello")          # {"nonce", "ciphertext", "tag"}
decrypt(key, box["nonce"], box["ciphertext"], box["tag"]) # b"hello"
```

Tampered ciphertext or a wrong key raises `DecryptionError` rather than returning garbage plaintext. `crypto\_engine/dh\_exchange.py` — ECDH (X25519) key exchange + HKDF-SHA256 key derivation:

```
from crypto_engine.dh_exchange import generate_keypair, compute_shared_key

priv_a, pub_a = generate_keypair()
priv_b, pub_b = generate_keypair()
assert compute_shared_key(priv_a, pub_b) == compute_shared_key(priv_b, pub_a)
```

`auth/password\_hash.py` — PBKDF2-HMAC-SHA256 password hashing, 200,000+ iterations, a fresh random salt per password, and a constant-time comparison (`hmac.compare\_digest`) on verify:

```
from auth.password_hash import hash_password, verify_password

stored = hash_password("correct horse battery staple")
# stored == "pbkdf2_sha256$200000$<salt_b64>$<hash_b64>"
assert verify_password("correct horse battery staple", stored)
assert not verify_password("wrong guess", stored)
```

`server/user\_store.py` persists only that encoded hash string per username, in `data/users.json` — the plaintext password is never written to disk. `crypto\_engine/signatures.py` — RSA-2048 signatures, RSA-PSS padding + SHA-256, with PEM (de)serialization helpers and the `fingerprint()` helper:

```
from crypto_engine.signatures import generate_keypair, sign, verify, fingerprint, serialize_public_key

private_key, public_key = generate_keypair()
signature = sign(private_key, b"hello")
assert verify(public_key, b"hello", signature) is True
assert verify(public_key, b"tampered", signature) is False # never raises
print(fingerprint(serialize_public_key(public_key))) # e.g. "A1B2 C3D4 E5F6 A7B8"
```

`auth/keystore.py` saves each account's RSA private key locally as an unencrypted PEM file under `data/keys/\_private.pem` (gitignored) — see "Known limitations".

`certs/generate\_certs.py` generates the self-signed TLS cert + key used by `server.py`/`client.py`, using the `cryptography` library directly (not the `openssl` CLI), so it produces an identical result on every teammate's machine:

```
python certs/generate_certs.py          # writes certs/server.key + certs/server.crt
python certs/generate_certs.py --force  # regenerate, overwriting existing certs
```

▼ GUI chat client — how it's built

`gui/chat\_gui.py` is a Tkinter GUI over the exact same client logic the terminal client uses — `client/client.py`'s `SecureChatClient`, `connect\_tls`, and the signature/keystore helpers are imported and reused directly; the GUI contains no protocol, crypto, or networking logic of its own. Besides a normal-looking chat window (dark theme, WhatsApp-style message bubbles), it has a second **Wire Log** panel that shows a live, timestamped feed of exactly what's crossing the network — handshake public keys and signatures, each message's nonce/ciphertext/tag (truncated for readability), and any rejected/tampered message in red. Passwords are redacted in the wire log with a note that TLS (not the GUI) is what actually protects them in transit; chat plaintext never appears there at all, only its ciphertext. **On the refactor this required:** `SecureChatClient`'s logic methods (register/login, handshake, send/receive) were already separate from the terminal-only presentation code (`authenticate()`/`main()`), which use `input()`/`print()` — the GUI calls `SecureChatClient` directly rather than those two functions, so no duplication of the protocol was needed. What *was* mixed into `SecureChatClient` itself was `print()` calls inside its own logic methods. Rather than remove those (risking a subtle change to the terminal client's exact behavior), an optional structured `event\_callback` was added to `SecureChatClient.\_\_init\_\_`: every one of those `print()` sites also emits a `(kind, data)` event through it, if one was supplied. The terminal client passes none, so it's unaffected. The GUI's callback only ever does `queue.Queue.put(...)` (thread-safe), and drains that queue on a `root.after()` timer to do all actual widget updates on Tkinter's main thread — `SecureChatClient`'s background `receive\_loop` thread never touches a widget directly.